Remote Workforce Identity Verification System

Proposed Solution and Technical Architecture

Version 1.0 | October 25, 2026 | Draft for internal review

Executive Summary

This document proposes a system that verifies that the person operating a remote employee's laptop is the same individual who was hired and authorized to work on it, independent of Microsoft Teams, Zoom, or meeting attendance. A lightweight Windows desktop agent performs webcam-based identity checks against an approved reference image managed in a central administrative console. Checks are transparent to employees, randomized within configured work windows, and verified server-side using AWS Rekognition.

The design combines four safeguards: **face matching** (one-to-one comparison against the enrolled reference image), **liveness detection** (an interactive challenge that defeats photos and prerecorded video), **device binding** (each installation is registered to one employee and one machine), and **human review** (uncertain results are flagged as CHECK, never treated as automatic proof of misconduct).

The recommended first release is a Windows agent plus a centralized web admin console. The architecture is deliberately structured so a macOS agent can be added later without reworking the backend, portal, or verification services.

1. Objective and Scope

Primary goal: establish auditable confidence that the enrolled employee is the person using the assigned computer during work sessions, while clearly distinguishing identity mismatches from technical failures and situations that simply require human review.

The initial release (MVP) covers:

- Secure administrator login, employee enrollment, and reference-image capture or upload.
- A signed Windows installer; device registration binding each installation to one employee and device.
- Webcam-based face detection and one-to-one identity matching with configurable thresholds.
- Interactive liveness verification at session start and on escalation.
- Randomized periodic checks during configured work windows, with a visible prompt and camera indicator.
- MATCH / CHECK / NO FACE / MULTIPLE FACES / VERIFICATION ERROR outcomes with full audit trail.
- A centralized dashboard with verification history, flagged-event review, alerts, and CSV export.
- Secure data handling, role-based access control, retention policies, and employee offboarding.

Out of scope for the first release: continuous video recording or monitoring, macOS deployment (architecture-ready, delivered in a later phase), advanced passive anti-spoofing vendors, and enterprise MDM integrations beyond standard MSI/MSIX deployment.

2. How Verification Works

Two complementary verification paths are used. This separation matters: AWS Face Liveness is an interactive challenge that requires the employee's attention, so it cannot run silently, and it costs roughly fifteen times more per check than a plain face comparison.

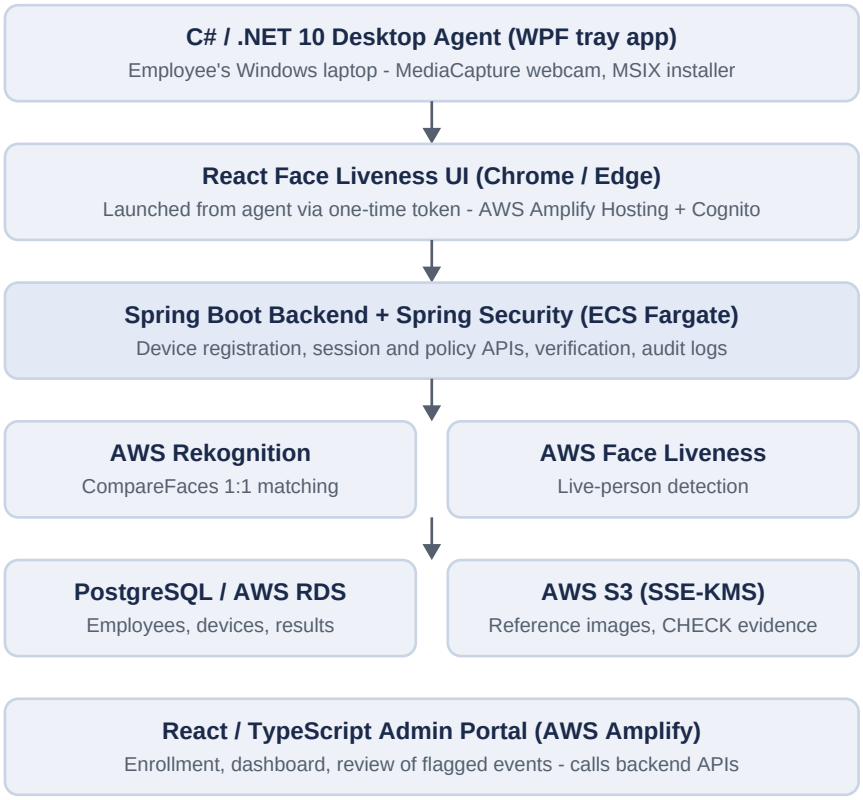
Path	When it runs	How it works
Active liveness check	Work-session start, and escalation after repeated CHECK results	The agent opens a hosted React page in Chrome/Edge via a one-time token. The employee completes the on-screen liveness challenge; the backend creates the session, retrieves the result and audit image, and runs CompareFaces against the reference image.
Passive periodic check	Random times within the configured work window (e.g. up to 4 checks between 9:00 and 17:00)	The agent shows a visible prompt, captures a single webcam frame, and uploads it over HTTPS with a server-issued one-time check ID. The backend runs DetectFaces (quality, face count) and CompareFaces, then records the outcome.

Verification outcomes

Status	Meaning	Required action
MATCH	Captured face meets the configured similarity threshold and image-quality requirements.	Record the successful verification with timestamp. The captured frame is discarded; only score and metadata are retained.
CHECK	Identity cannot be confirmed, the score is borderline, or review is required.	Flag for human review; prompt the employee to retry where appropriate. Never treated as automatic proof of misconduct.
NO FACE	No usable face detected in the capture.	Prompt the employee to position themselves in front of the camera and retry.
MULTIPLE FACES	More than one face detected where a single authorized user is expected.	Flag for review and request a new capture.
VERIFICATION ERROR	Camera, network, or verification-service failure.	Record the technical failure and retry per policy. Technical failure is never recorded as an identity mismatch.

Matching thresholds are configurable; the proposed starting policy is: similarity at or above 90 returns MATCH, 75-90 returns CHECK, and below 75 returns CHECK with a mismatch flag. Final thresholds are validated during acceptance testing across representative lighting conditions, camera hardware, and user populations, with measured false-acceptance and false-rejection rates.

3. System Architecture



All verification decisions are made server-side; the desktop agent never determines or reports its own verification result. The agent authenticates with per-device credentials issued at registration and stored in Windows Credential Manager (DPAPI), so every verification event is bound to a specific employee and a specific registered device.

4. Technology Stack

Component	Technology	Rationale / notes
Desktop agent	C# + .NET 10 (LTS), WPF tray application	Native Windows integration; runs in the user session (never as a Session 0 service, which cannot access the camera).
Webcam capture	Windows MediaCapture API	Native camera access; automatically triggers the Windows camera-in-use indicator, satisfying the transparency requirement.
Check prompts	Windows toast notifications	Visible, dismissible identity-check prompts with a response window.
Installer / updates	MSIX, code-signed	Clean install, upgrade, and removal; built-in auto-update via App Installer or Intune/MDM.
Agent credentials	Windows Credential Manager / DPAPI	Per-device secret issued at registration; encrypted at rest.
Scheduling	.NET PeriodicTimer + server-issued policy	Random check times with jitter inside employer-configured work windows; policy always comes from the server.

Component	Technology	Rationale / notes
Backend	Java Spring Boot on AWS ECS Fargate	REST APIs for registration, sessions, verification, policy, and audit.
Security / auth	Spring Security + JWT / OAuth 2.0; Amazon Cognito	Role-based admin access. Cognito Identity Pool is required by the Face Liveness browser component.
Face matching	AWS Rekognition CompareFaces + DetectFaces	Managed 1:1 matching with similarity scores; DetectFaces supplies face count and image-quality signals. Stateless: AWS does not retain faces from these calls.
Liveness detection	Amazon Rekognition Face Liveness	Interactive live-person challenge; defeats photos and prerecorded video.
Liveness UI	React + AWS Amplify UI FaceLivenessDetector, Amplify Hosting	Hosted verification page opened from the agent in Chrome/Edge (WebRTC required).
Admin portal	React.js + TypeScript on AWS Amplify	Enrollment, dashboard, review workflows, exports.
Database	PostgreSQL on AWS RDS (+ Flyway migrations)	Employees, devices, policies, verification events, audit log.
Image storage	AWS S3 with SSE-KMS and lifecycle rules	Reference images and CHECK-event evidence only; lifecycle rules enforce retention policy.
API contract	OpenAPI (springdoc)	Formal, platform-neutral agent-to-backend specification.
Monitoring	AWS CloudWatch	Logs, metrics, alarms.
CI/CD	GitHub Actions	Build, test, sign, and deploy pipelines.

Region constraint: Rekognition Face Liveness is available in a limited set of AWS regions (N. Virginia, Oregon, Ireland, Tokyo, Mumbai). All components will be deployed in one of these regions.

5. macOS Readiness

Everything server-side is platform-neutral and requires no change for macOS: the backend, Rekognition services, admin portal, and the browser-based liveness page (Safari and Chrome are both supported). Two design measures in the Windows build keep the macOS door open at near-zero cost:

- **Platform-neutral API contract:** device registration carries a platform field; device identifiers are server-issued and opaque; nothing in the API assumes Windows.
- **Layered agent:** all business logic (API client, scheduler, retry queue, check lifecycle) lives in a platform-neutral .NET core library behind small interfaces (camera, notifications, credential store). The WPF Windows head supplies only platform pieces; a future macOS head reuses the entire core.

The macOS phase itself will require: Apple Developer ID enrollment and notarization (mandatory for distribution), AVFoundation for camera capture, Keychain for credentials, a menu-bar app launched via LaunchAgent (user session only - a root daemon cannot access the camera), and Sparkle or MDM for updates.

6. Security, Privacy, and Anti-Tamper Design

Data protection

- TLS for all traffic; SSE-KMS encryption at rest for images; encrypted database storage.
- Data minimization: captured frames for MATCH results are discarded - only timestamp, score, and status are kept. Full captures are retained only for CHECK and flagged events, pending review, under configurable retention periods.
- Role-based access control in the admin portal; all access to biometric records is logged in a tamper-evident audit trail.
- Offboarding deletes or revokes biometric records, device credentials, and access, subject to retention obligations.

Transparency to employees

- Checks are disclosed in advance; the agent shows a visible prompt and the operating system's camera indicator - no covert capture.
- Written notice and, where required, consent are obtained before deployment; an alternative verification process is provided for employees who cannot use facial recognition or whose verification repeatedly fails.
- CHECK is a review workflow: verification results are never the sole basis for adverse employment decisions.

Anti-spoofing and anti-tamper

- Server-issued one-time check IDs with short upload windows prevent replaying old photos.
- Camera-device enumeration flags virtual webcams (e.g. OBS, ManyCam); anomalies escalate to a random-timed interactive liveness check.
- Interactive Face Liveness at session start defeats photos, screens, and prerecorded video.
- The agent cannot decide results locally; all verification and policy come from the server.
- Signed installer and signed updates; device authorization can be revoked centrally at any time.

Legal prerequisite: biometric and employment-law review in every deployment jurisdiction (e.g. GDPR and works-council requirements in the EU, Illinois BIPA in the US, India's DPDP Act) is a go/no-go gate before production rollout, not a parallel activity.

7. Phased Delivery Plan

Phase	Scope	Outcome
Phase 1 - MVP	Backend, admin portal, enrollment, device registration, Windows agent with passive randomized checks (DetectFaces + CompareFaces), all five statuses, dashboard, audit log, CSV export.	End-to-end verification with human review; satisfies most MVP acceptance criteria.
Phase 2 - Liveness	Hosted Face Liveness page, one-time-token browser flow from the agent, session-start and escalation liveness checks, Cognito integration.	Strong anti-spoofing; photos and video replay defeated.
Phase 3 - Hardening & macOS	Anti-tamper hardening (watchdog), alert rules, reporting polish, accuracy/bias validation campaign, macOS agent build, MDM integration as needed.	Production-grade operation and cross-platform coverage.

8. Prerequisites

- **Legal and consent review** in all deployment jurisdictions, including the documented alternative-verification process (launch blocker).
- **AWS account** in a Face Liveness-supported region; IAM roles, Cognito Identity Pool, budget alarms.
- **Code-signing certificate** for the Windows installer (longest procurement lead time after legal; Azure Trusted Signing or EV certificate).
- **Decided retention periods** for reference images, verification captures, and audit records - these drive the S3 lifecycle configuration and database schema.
- **Deployment model decision:** company-managed devices vs. employee-owned (BYOD) - this determines deployment tooling, tamper-resistance assumptions, and consent posture.
- **Threshold policy** agreed and later validated against measured false-acceptance/false-rejection criteria.
- **Test-hardware matrix:** three to four representative laptop models with different webcams.

9. Indicative Operating Costs

Ballpark monthly AWS costs for approximately 100 employees (22 working days, one liveness check per day, passive checks every 30 minutes across an 8-hour window):

Item	Basis	Approx. monthly cost
Face Liveness checks	~2,200 checks at ~$0.015 per check	~$33
CompareFaces + DetectFaces	~70,000 image analyses at ~$0.001 per image	~$70
Infrastructure	ECS Fargate, RDS, ALB, NAT, S3, CloudWatch	~$150-250
Total		~$250-350 / month

Figures are indicative, based on published per-unit rates, and exclude development effort, code-signing and Apple Developer fees, and support. Rekognition is not the cost driver; infrastructure is. Confirm current rates on the official AWS pricing pages before budgeting.

10. Assurance Level and Known Limitations

Facial verification establishes that the enrolled employee was present at the camera at the moments of verification. It does not, by itself, prove who performed every keystroke or work activity between checks. The combination of device binding, randomized check times, liveness challenges, and an auditable event history raises the practical difficulty of impersonation substantially, but the evidence should be communicated and used with this limitation in mind.

- A CHECK result is a request for review, never a verdict. Poor lighting, camera failure, or a borderline score must not be treated as fraud.
- Passive snapshot checks alone are weaker against sophisticated spoofing (e.g. virtual cameras); the interactive liveness path and device-enumeration flags are the backstop.
- Accuracy must be validated across lighting conditions, camera types, and skin tones before the system is relied upon; measured FAR/FRR are an acceptance gate.
- On employee-owned machines, anti-tamper guarantees are inherently limited; company-managed devices are recommended for stronger assurance.

End of proposal. Supporting artifacts to follow in Phase 1: repository layout, OpenAPI specification for the agent-to-backend API, and the PostgreSQL schema.